indebted for the editing of the famous "Debian Free Software Guidelines" (DFSG), an original idea of Ean Schuessler. Subsequently, Bruce would derive from it the famous "Open Source Definition", removing all references to Debian from it.

➨ https://opensource.org/

His departure from the project was quite emotional, but Bruce has remained strongly attached to Debian, since he continues to promote this distribution in political and economic spheres. He still sporadically appears on the e-mail lists to give his advice and present his latest initiatives in favor of Debian.

Last anecdotal point, it was Bruce who was responsible for inspiring the different "codenames" for Debian versions (1.1 — *Rex*, 1.2 — *Buzz*, 1.3 — *Bo*, 2.0 — *Hamm*, 2.1 — *Slink*, 2.2 — *Potato*, 3.0 — *Woody*, 3.1 — *Sarge*, 4.0 — *Etch*, 5.0 — *Lenny*, 6.0 — *Squeeze*, 7 — *Wheezy*, 8 — *Jessie*, 9 — *Stretch*, 10 — *Buster*, 11 (not released yet) — *Bullseye*, 12 (not released yet) — *Bookworm*, *Unstable* — *Sid*). They are taken from the names of characters in the Toy Story movie. This animated film entirely composed of computer graphics was produced by Pixar Studios, with whom Bruce was employed at the time that he led the Debian project. The name "Sid" holds particular status, since it will eternally be associated with the *Unstable* branch. In the film, this character was the neighbor's child who always broke toys — so beware of getting too close to *Unstable*. Otherwise, *Sid* is also an acronym for "Still In Development".

1.3. The Inner Workings of the Debian Project

The abundant end results produced by the Debian project derive simultaneously from the work on the infrastructure performed by experienced Debian developers, from the individual or collective work of developers on Debian packages, and from user feedback.

1.3.1. The Debian Developers

Debian developers have various responsibilities, and as official project members, they have great influence on the direction the project takes. A Debian developer is generally responsible for at least one package, but according to their available time and desire, they are free to become involved in numerous teams, thus acquiring more responsibilities within the project.

➨ https://www.debian.org/devel/people

➨ https://www.debian.org/intro/organization

➨ https://wiki.debian.org/Teams

TOOL

Developer's database

Debian has a database including all developers registered with the project, and their relevant information (address, telephone, geographical coordinates such as longitude and latitude, etc.). Some of the information (first and last name, country, username within the project, IRC username, GnuPG key, etc.) is public and available on the Web.

➨ https://db.debian.org/

The geographical coordinates allow the creation of a map locating all of the developers around the globe. Debian is truly an international project: its developers can be found on all continents, although the majority are in "Western countries".

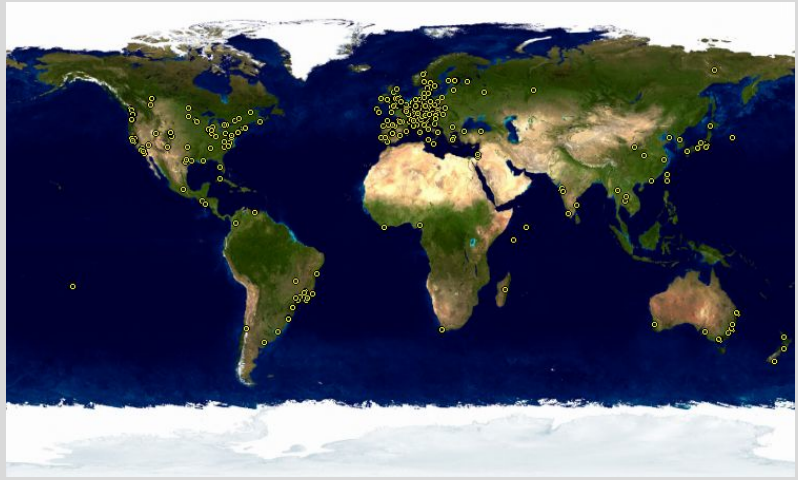


Figure 1.1 *World-wide distribution of Debian developers*

Package maintenance is a relatively regimented activity, very documented or even regulated. It must, in effect, comply with all the standards established by the *Debian Policy*. Fortunately, there are many tools that facilitate the maintainer's work. The developer can, thus, focus on the specifics of their package and on more complex tasks, such as squashing bugs.

➨ `https://www.debian.org/doc/debian-policy/`

BACK TO BASICS

Package maintenance, the developer's work

Maintaining a package entails, first, "packaging" a program. Specifically, this means to define the means of installation so that, once installed, this program will operate and comply with the rules which the Debian project sets for itself. The result of this operation is saved in a `.deb` file. Effective installation of the program will then require nothing more than extraction of this compressed archive and execution of some pre-installation or post-installation scripts contained therein.

After this initial phase, the maintenance cycle truly begins: preparing updates to follow the latest version of the Debian Policy, fixing bugs reported by users, and including new "upstream" versions of the program which naturally continues to develop simultaneously. For instance, at the time of the initial packaging, the program was at version 1.2.3. After some months of development, the original authors release a new stable version, numbered 1.4.0. At this point, the Debian maintainer should update the package, so that users can benefit from its latest stable version.

The Policy, an essential element of the Debian Project, establishes the norms ensuring both the quality of the packages and perfect interoperability of the distribution. Thanks to this Policy, Debian remains consistent despite its gigantic size. This Policy is not fixed in stone, but continuously evolves thanks to proposals formulated on the debian-policy@lists.debian.org mailing

list. Amendments that are agreed upon by all interested parties are accepted and applied to the text by a small group of maintainers who have no editorial responsibility (they only include the modifications agreed upon by the Debian developers that are members of the above-mentioned list). You can read current amendment proposals on the bug tracking system:

➨ https://bugs.debian.org/debian-policy

COMMUNITY

Policy editorial process

Anyone can propose an amendment to the Debian Policy just by submitting a bug report with a severity level of "wishlist" against the *debian-policy* package. The process that then starts is documented in https://www.debian.org/doc/debian-policy/ap-process.html: if it is acknowledged that the problem revealed must be resolved by creating a new rule in the Debian Policy, a discussion begins on the debian-policy@lists.debian.org mailing list until consensus is reached and a proposal issued. Someone then drafts a desired amendment and submits it for approval (in the form of a patch to review). As soon as two other developers approve the fact that the proposed amendment reflects the consensus reached in the previous discussion (they "second" it), the proposal can be included in the official document by one of the *debian-policy* package maintainers. If the process fails at one of these steps, the maintainers close the bug, classifying the proposal as rejected.

DEBIAN POLICY

The documentation

Documentation for each package is stored in `/usr/share/doc/package/`. This directory often contains a `README.Debian` file describing the Debian specific adjustments made by the package maintainer. It is, thus, wise to read this file prior to any configuration, in order to benefit from their experience. We also find a `changelog.Debian.gz` file describing the changes made from one version to the next by the Debian maintainer. This is not to be confused with the `changelog.gz` file (or equivalent), which describes the changes made by the upstream developers. The `copyright` file includes information about the authors and the license covering the software. Finally, we may also find a file named `NEWS.Debian.gz`, which allows the Debian developer to communicate important information regarding updates; if *apt-listchanges* is installed, then these messages are automatically displayed. All other files are specific to the software in question. We especially would like to point out the `examples` sub-directory, which frequently contains examples of configuration files.

The Policy provides considerable cover of the technical aspects of packaging. The size of the project also raises organizational problems; these are dealt with by the Debian Constitution, which establishes a structure and means for decision making. In other words, a formal governance system.

This constitution defines a certain number of roles and positions, plus responsibilities and authorities for each. It is particularly worth noting that Debian developers always have ultimate decision making authority by a vote of general resolution, wherein a qualified majority of three quarters (75%) of votes is required for significant alterations to be made (such as those with an impact on the Foundation Documents). However, developers annually elect a "leader" to represent them in meetings, and ensure internal coordination between varying teams. This election is always a period of intense discussions. This leader's role is not formally defined by any document: candidates for this post usually propose their own definition of the position. In practice,

the leader's roles include serving as a representative to the media, coordinating between "internal" teams, and providing overall guidance to the project, within which the developers can relate: the views of the DPL are implicitly approved by the majority of project members.

Specifically, the leader has real authority; their vote resolves tie votes; they can make any decision which is not already under the authority of someone else and can delegate part of their responsibilities.

Since its inception, the project has been successively led by Ian Murdock, Bruce Perens, Ian Jackson, Wichert Akkerman, Ben Collins, Bdale Garbee, Martin Michlmayr, Branden Robinson, Anthony Towns, Sam Hocevar, Steve McIntyre, Stefano Zacchiroli, Lucas Nussbaum, Mehdi Dogguy, Chris Lamb and Sam Hartman.

The constitution also defines a "technical committee". This committee's essential role is to decide on technical matters when the developers involved have not reached an agreement between themselves. Otherwise, this committee plays an advisory role for any developer who fails to make a decision for which they are responsible. It is important to note that they only get involved when invited to do so by one of the parties in question.

Finally, the constitution defines the position of "project secretary", who is in charge of the organization of votes related to the various elections and general resolutions.

The "general resolution" procedure is fully detailed in the constitution, from the initial discussion period to the final counting of votes. The most interesting aspect of that process is that when it comes to an actual vote, developers have to rank the different ballot options between them and the winner is selected with a Condorcet method[1] (more specifically, the Schulze method). For further details see:

➨ https://www.debian.org/devel/constitution

CULTURE

Flamewar, the discussion that catches fire

A "flamewar" is an exceedingly impassioned debate, which frequently ends up with people attacking each other once all reasonable argumentation has been exhausted on both sides. Certain themes are more frequently subject to polemics than others (the choice of text editor, "do you prefer `vi` or `emacs`?", is an old favorite). The matters often provoke very rapid e-mail exchanges due to the sheer number of people with an opinion on the matter (everyone) and the very personal nature of such questions.

Nothing particularly useful generally comes from such discussions; the general recommendation is to stay out of such debates, and maybe rapidly skim through their content, since reading them in full would be too time-consuming.

Even if this constitution establishes a semblance of democracy, the daily reality is quite different: Debian naturally follows the free software rules of the do-ocracy: the one who does things gets to decide how to do them. A lot of time can be wasted debating the respective merits of various ways to approach a problem; the chosen solution will be the first one that is both functional and satisfying... which will come out of the time that a competent person did put into it.

[1] https://en.wikipedia.org/wiki/Condorcet_method

This is the only way to earn one's stripes: do something useful and show that one has worked well. Many Debian "administrative" teams operate by co-optation, preferring volunteers who have already effectively contributed and proved their competence. The public nature of the work of those teams makes it possible for new contributors to observe and start helping without any special privilege. This is why Debian is often described as a "meritocracy".

CULTURE

Meritocracy, the reign of knowledge

Meritocracy is a form of government in which authority is exercised by those with the greatest merit. For Debian, merit is a measure of competence, which is, itself, assessed by observation of past actions by one or more others within the project (Stefano Zacchiroli, a former project leader, speaks of "do-ocracy", meaning "power to those who get things done"). Their simple existence proves a certain level of competence; their achievements generally being free software, with available source code, which can easily be reviewed by peers to assess their quality.

This effective operational method guarantees the quality of contributors in the "key" Debian teams. This method is by no means perfect and occasionally there are those who do not accept this way of operating. The selection of developers accepted in the teams may appear a bit arbitrary, or even unfair. Furthermore, not everybody has the same definition of the service expected from these teams. For some, it is unacceptable to have to wait eight days for inclusion of a new Debian package, while others will wait patiently for three weeks without a problem. As such, there are regular complaints from the disgruntled about the "quality of service" from some teams.

COMMUNITY

Integration of new maintainers

The team in charge of admitting new developers is the most regularly criticized. One must acknowledge that, throughout the years, the Debian project has become more and more demanding of the developers that it will accept. Some people may see some injustice in that, but we must confess that what were only little challenges at the beginning have become much greater in a community of over 1,000 people, when it comes to ensuring the quality and integrity of everything that Debian produces for its users.

Furthermore, the acceptance procedure is concluded by review of the candidacy by a small team, the Debian Account Managers. These managers are, thus, particularly exposed to criticism, since they have final say in the inclusion or rejection of a volunteer within the Debian developers community. In practice, sometimes they must delay the acceptance of a person until they have learned more about the operations of the project. One can, of course, contribute to Debian before being accepted as an official developer, by being sponsored by current developers.

1.3.2. The Active Role of Users

One might wonder if it is relevant to mention the users among those who work within the Debian project, but the answer is a definite yes: they play a critical role in the project. Far from being "passive", some users run development versions of Debian and regularly file bug reports to indicate problems. Others go even further and submit ideas for improvements, by filing a bug report with a severity level of "wishlist", or even submit corrections to the source code, called "patches" (see section 1.3.2.3, "Sending fixes" page 15).

The fundamental tool for submitting bugs in Debian is the Debian Bug Tracking System (Debian BTS), which is used by large parts of the project. The public part (the web interface) allows users to view all bugs reported, with the option to display a sorted list of bugs selected according to various criteria, such as: affected package, severity, status, address of the reporter, address of the maintainer in charge of it, tag, etc. It is also possible to browse the complete historical listing of all discussions regarding each of the bugs.

Below the surface, the Debian BTS is e-mail based: all information that it stores comes from messages sent by the various persons involved. Any e-mail sent to 12345@bugs.debian.org will, thus, be assigned to the history for bug number 12345. Authorized persons may "close" a bug by writing a message describing the reasons for the decision to close to 12345-done@bugs.debian.org (a bug is closed when the indicated problem is resolved or no longer relevant). A new bug is reported by sending an e-mail to submit@bugs.debian.org according to a specific format which identifies the package in question. The address control@bugs.debian.org allows editing of all the "meta-information" related to a bug.

The Debian BTS has other functional features, as well, such as the use of tags for labeling bugs. For more information, see

➨ `https://www.debian.org/Bugs/`

VOCABULARY

Severity of a bug

The severity of a bug formally assigns a degree of gravity to the reported problem. Effectively, not all bugs have the same importance; for instance, a typo in a manual page is not comparable to a security vulnerability in server software.

Debian uses an extended scale to describe the severity of a bug. Each level is defined precisely in order to facilitate the selection thereof.

➨ `https://www.debian.org/Bugs/Developer#severities`

Users can also use the command line to send bug reports on a Debian package with the `reportbug` tool. It helps making sure the bug in question hasn't already been filed, thus preventing redundancy in the system. It reminds the user of the definitions of the severity levels, for the report to be as accurate as possible (the developer can always fine-tune these parameters later, if needed). It helps writing a complete bug report without the user needing to know the precise syntax, by writing it and allowing the user to edit it. This report will then be sent via an e-mail server (by default, a remote one run by Debian, but `reportbug` can also use a local server).

This tool first targets the development versions, which is where the bugs will be fixed. Effectively, changes are not welcome in a stable version of Debian, with very few exceptions for security updates or other important updates (if, for example, a package is not working at all). A correction of a minor bug in a Debian package must, thus, wait for the next stable version.

Translation and documentation

Additionally, numerous satisfied users of the service offered by Debian like to make a contribution of their own to the project. As not everyone has appropriate levels of expertise in programming, they may choose to assist with the translation and review of documentation. There are language-specific mailing lists to coordinate this work.

➡ https://lists.debian.org/i18n.html

➡ https://www.debian.org/international/

BACK TO BASICS

What are i18n and l10n?

"i18n" and "l10n" are the abbreviations for the words "internationalization" and "localization", respectively, preserving the initial and last letter of each word, and the number of letters in the middle.

To "internationalize" a program consists of modifying it so that it can be translated (localized). This involves partially rewriting a program initially written to work in one language in order to be able to open it to all languages.

To "localize" a program consists of translating the original messages (frequently in English) to another language. For this, it must have already been internationalized.

In summary, internationalization prepares the software for translation, which is then executed by localization.

Sending fixes

More advanced users might be able to provide a fix to a program by sending a patch.

A patch is a file describing changes to be made to one or more reference files. Specifically, it will contain a list of lines to be removed or added to the code, as well as (sometimes) lines taken from the reference text, replacing the modifications in context (they allow identification of the placement of the changes if the line numbers have changed).

The tool used for applying the modifications given in such a file is simply called `patch`. The tool that creates it is called `diff`, and is used as follows:

```
$ diff -u file.old file.new >file.patch
```

The `file.patch` file contains the instructions for changing the content of `file.old` into `file.new`. We can send it to someone, who can then use it to recreate `file.new` from the two others, like this:

```
$ patch -p0 file.old <file.patch
```

The file, `file.old`, is now identical to `file.new`.

In practice, most software is maintained in Git repositories and contributors are thus more likely to use `git` to retrieve the source code and propose changes. `git diff` will generate a file in the same format as what `diff -u` would do and `git apply` can do the same as `patch`.

CULTURE

Git

Git is a tool for collaborative work on multiple files, while maintaining a history of modifications. The files in question are generally text files, such as a program's source code. If several people work together on the same file, `git` can only merge the alterations made if they were made to different portions of the file. Otherwise, these "conflicts" must be resolved by hand.

Git is a distributed system where each user has a repository with the complete history of changes. Central repositories are used to download the project (`git clone`) and to share the work done with others (`git push`). The repository can contain multiple versions of the files but only one version can be worked on at a given time: it is called the working copy (it can be changed to point to another version with `git checkout`). Git can show you the modifications made to the working copy (`git diff`), can stage changes for inclusion (`git add`)), and can create a new entry in the versions history (`git commit`). It can also update the working copy to include modifications made in parallel by other users (`git pull`), and can record a particular configuration in the history in order to be able to easily extract it later on (`git tag`).

Git makes it easy to handle multiple concurrent versions of a project in development without them interfering with each other. These versions are called *branches*. This metaphor of a tree is fairly accurate, since a program is initially developed on a common trunk. When a milestone has been reached (such as version 1.0), development continues on two branches: the development branch prepares the next major release, and the maintenance branch manages updates and fixes for version 1.0.

Git is, nowadays, the most popular version control system but it is not the only one. Historically, CVS (Concurrent Versions System) was the first widely used tool but its numerous limitations contributed to the appearance of more modern free alternatives. These include, especially, `subversion` (`svn`), `git`, `bazaar` (`bzr`), and `mercurial` (`hg`).

➨ `https://www.nongnu.org/cvs/`

➨ `https://subversion.apache.org/`

➨ `https://git-scm.com/`

➨ `https://bazaar.canonical.com/`

➨ `http://mercurial.selenic.com/`

It is beyond the scope of this book to provide a detailed explanation about Git, for that you can refer to the *Pro Git* book.

➨ `https://git-scm.com/book/`

While the output of `git diff` is a file that can be shared with other developers, there are usually better ways to submit changes. If the developers prefer to get patches by email, they usually want patches generated with `git format-patch` so that they can be directly integrated in the repository with `git am`. This preserves commits meta-information and makes it possible to share multiple commits at once.

This email-based workflow is still popular but it tends to be replaced by the usage of *merge requests* (or *pull requests*) whenever the software is hosted in a platform like Github or GitLab — and Debian is using GitLab on its salsa.debian.org server. On those systems, once you have created an account, you *fork* the repository, effectively creating a copy of the repository in your own

account, and you can then clone that repository and push your own changes in it. From there, the web interface will suggest you to submit a merge request, notifying the developers of your changes, making it easy for them to review and accept your changes with a single click.

Other ways of contributing

All of these contribution mechanisms are made more efficient by users' behavior. Far from being a collection of isolated persons, users are a true community within which numerous exchanges take place. We especially note the impressive activity on the user discussion mailing list, debian-user@lists.debian.org (chapter 7, "Solving Problems and Finding Relevant Information" page 148 discusses this in greater detail).

➨ `https://lists.debian.org/users.html`

Not only do users help themselves (and others) on technical issues that directly affect them, but they also discuss the best ways to contribute to the Debian project and help it move forward — discussions that frequently result in suggestions for improvements.

TOOL

how-can-i-help

The `how-can-i-help` program lists opportunities for contributing to Debian packages that are installed locally. After each APT invocation, it shows ways to help, by highlighting bugs tagged "newcomer" (which are easy entry-points for new contributors) or orphaned packages that need a new maintainer. The program can be executed directly as well.

Since Debian does not expend funds on any self-promoting marketing campaigns, its users play an essential role in its diffusion, ensuring its fame via word-of-mouth.

This method works quite well, since Debian fans are found at all levels of the free software community: from install parties (workshops where seasoned users assist newcomers to install the system) organized by local LUGs or "Linux User Groups", to association booths at large tech conventions dealing with Linux, etc.

Volunteers make posters, brochures, stickers, and other useful promotional materials for the project, which they make available to everyone, and which Debian provides freely on its website and on its wiki:

➨ `https://www.debian.org/events/material`

1.3.3. Teams and Sub-Projects

Debian has been organized, right from the start, around the concept of source packages, each with its maintainer or group of maintainers. Many work teams have emerged over time, ensuring administration of the infrastructure, management of tasks not specific to any package in particular (quality assurance, Debian Policy, installer, etc.), with the latest series of teams growing up around sub-projects.

Existing Debian Sub-Projects

To each their own Debian! A sub-project is a group of volunteers interested in adapting Debian to specific needs. Beyond the selection of a sub-group of programs intended for a particular domain (education, medicine, multimedia creation, etc.), sub-projects are also involved in improving existing packages, packaging missing software, adapting the installer, creating specific documentation, and more.

VOCABULARY

Sub-project and derivative distribution

The development process for a derivative distribution consists in starting with a particular version of Debian and making a number of modifications to it. The infrastructure used for this work is completely external to the Debian project. There isn't necessarily a policy for contributing improvements. This difference explains how a derivative distribution may "diverge" from its origins, and why they have to regularly resynchronize with their source in order to benefit from improvements made upstream.

On the other hand, a sub-project can not diverge, since all the work on it consists of directly improving Debian in order to adapt it to a specific goal.

The most known distribution derived from Debian is, without a doubt, Ubuntu, but there are many. See appendix A, "Derivative Distributions" page 469 to learn about their particularities and their positioning in relationship to Debian.

Here is a small selection of current sub-projects:

- Debian Jr., by Ben Armstrong, offering an appealing and easy to use Debian system for children;
- Debian Edu, by Petter Reinholdtsen, focused on the creation of a specialized distribution for the academic world;
- Debian Med, by Andreas Tille, dedicated to the medical field;
- Debian Multimedia which deals with audio and multimedia work;
- Debian GIS which takes care of Geographical Information Systems applications and users;
- Debian Accessibility, improving Debian to match the requirements of people with disabilities;
- Debian Science, finally, working on providing researchers and scientists a better experience using Debian.
- DebiChem, targeted at Chemistry, provides chemical suites and programs.

The number of projects will most likely continue to grow with time and improved perception of the advantages of Debian sub-projects. Fully supported by the existing Debian infrastructure, they can, in effect, focus on work with real added value, without worrying about remaining synchronized with Debian, since they are developed within the project.

Administrative Teams

Most administrative teams are relatively closed and recruit only by co-optation. The best means to become a part of one is to intelligently assist the current members, demonstrating that you have understood their objectives and methods of operation.

The *ftpmasters* are in charge of the official archive of Debian packages. They maintain the program that receives packages sent by developers and automatically stores them, after some checks, on the reference server (ftp-master.debian.org).

They must also verify the licenses of all new packages, in order to ensure that Debian may distribute them, prior to including them in the corpus of existing packages. When a developer wishes to remove a package, they address this team through the bug tracking system and the *ftp.debian.org* "pseudo-package".

VOCABULARY

The pseudo-package, a monitoring tool

The bug tracking system, initially designed to associate bug reports with a Debian package, has proved very practical to manage other matters: lists of problems to be resolved or tasks to manage without any link to a particular Debian package. The "pseudo-packages" allow, thus, certain teams to use the bug tracking system without associating a real package with their team. Everyone can, thus, report issues that needs to be dealt with. For instance, the BTS has a *ftp.debian.org* entry that is used to report and track problems on the official package archive or simply to request removal of a package. Likewise, the *www.debian.org* pseudo-package refers to errors on the Debian website, and *lists.debian.org* gathers all the problems concerning the mailing lists.

TOOL

GitLab, Git repository hosting and much more

A GitLab instance, known as `salsa.debian.org`, is used by Debian to host the Git packaging repositories but this software offers much more than simple hosting and Debian contributors have been quick to leverage the continuous integration features (running tests, or even building packages, on each push). Debian contributors also benefit from a cleaner contribution workflow thanks the well understood merge request process (similar to GitHub's pull requests).

GitLab replaced FusionForge (which was running on a service known as `alioth.debian.org`) for collaborative package maintenance. This service is administered by Alexander Wirt, Bastian Blank and Jörg Jaspert.

➨ `https://salsa.debian.org/`

➨ `https://wiki.debian.org/Salsa/Doc`

The *Debian System Administrators* (DSA) team (debian-admin@lists.debian.org), as one might expect, is responsible for system administration of the many servers used by the project. They ensure optimal functioning of all base services (DNS, Web, e-mail, shell, etc.), install software requested by Debian developers, and take all precautions in regards to security.

➨ `https://dsa.debian.org`

TOOL

Debian Package Tracker

This is one of Raphaël's creations. The basic idea is, for a given package, to centralize as much information as possible on a single page. Thus, one can quickly check the status of a program, identify tasks to be completed, and offer one's assistance. This is why this page gathers all bug statistics, available versions in each distribution, progress of a package in the *Testing* distribution, the status of translations of descriptions and debconf templates, the possible availability of a new upstream version, notices of noncompliance with the latest version of the Debian Policy, information on the maintainer, and any other information that said maintainer wishes to include.

➨ `https://tracker.debian.org/`

An e-mail subscription service completes this web interface. It automatically sends the following selected information to the list: bugs and related discussions, availability of a new version on the Debian servers, new translations available for proofreading, etc.

Advanced users can, thus, follow all of this information closely and even contribute to the project, once they have got a good enough understanding of how it works.

Another web interface, known as *Debian Developer's Packages Overview* (DDPO), provides each developer a synopsis of the status of all Debian packages placed under their charge.

➨ `https://qa.debian.org/developer.php`

These two websites are tools developed and managed by the group responsible for quality assurance within Debian (known as Debian QA).

The *listmasters* administer the e-mail server that manages the mailing lists. They create new lists, handle bounces (delivery failure notices), and maintain spam filters (unsolicited bulk e-mail).

CULTURE

Traffic on the mailing lists: some figures

The mailing lists are, without a doubt, the best testimony to activity on a project, since they keep track of everything that happens. The numbers (from May 2019) regarding our mailing lists speak for themselves: Debian hosts about 315 lists, totaling over 303,000 individual subscriptions. 227,000 e-mails are delivered every day.

Each specific service has its own administration team, generally composed of volunteers who have installed it (and also frequently programmed the corresponding tools themselves). This is the case of the bug tracking system (BTS), the package tracker, salsa.debian.org (GitLab server, see sidebar "GitLab, Git repository hosting and much more" page 19), the services available on qa.debian.org, lintian.debian.org, buildd.debian.org, cdimage.debian.org, etc.

Development Teams, Transversal Teams

Unlike administrative teams, the development teams are rather widely open, even to outside contributors. Even if Debian does not have a vocation to create software, the project needs some specific programs to meet its goals. Of course, developed under a free software license, these tools make use of methods proven elsewhere in the free software world.

Debian has developed little software of its own, but certain programs have assumed a starring role, and their fame has spread beyond the scope of the project. Good examples are `dpkg`, the Debian package management program (it is, in fact, an abbreviation of Debian PacKaGe, and generally pronounced as "dee-package"), and `apt`, a tool to automatically install any Debian package, and its dependencies, guaranteeing the consistency of the system after an upgrade (its name is an acronym for Advanced Package Tool). Their teams are, however, much smaller, since a rather high level of programming skill is required to gain an overall understanding of the operations of these types of programs.

The most important team is probably that for the Debian installation program, `debian-installer`, which has accomplished a work of momentous proportions since its conception in 2001. Numerous contributors were needed, since it is difficult to write a single program able to install Debian on a dozen different architectures. Each one has its own mechanism for booting and its own bootloader. All of this work is coordinated on the debian-boot@lists.debian.org mailing list, under the direction of Cyril Brulebois.

➨ `https://www.debian.org/devel/debian-installer/`

➨ `https://joeyh.name/blog/entry/d-i_retrospective/`

The (very small) `debian-cd` program team has an even more modest objective. Many "small" contributors are responsible for their architecture, since the main developer can not know all the subtleties, nor the exact way to start the installer from the CD-ROM.

Many teams must collaborate with others in the activity of packaging: debian-qa@lists.debian.org tries, for example, to ensure quality at all levels of the Debian project. The debian-policy@lists.debian.org list develops Debian Policy according to proposals from all over the place. The teams in charge of each architecture (debian-*architecture*@lists.debian.org) compile all packages, adapting them to their particular architecture, if needed.

Other teams manage the most important packages in order to ensure maintenance without placing too heavy a load on a single pair of shoulders; this is the case with the C library and debian-glibc@lists.debian.org, the C compiler on the debian-gcc@lists.debian.org list, or Xorg on the debian-x@lists.debian.org (this group is also known as the X Strike Force).

1.4. Follow Debian News

As already mentioned, the Debian project evolves in a very distributed, very organic way. As a consequence, it may be difficult at times to stay in touch with what happens within the project without being overwhelmed with a never-ending flood of notifications.

If you only want the most important news about Debian, you probably should subscribe to the debian-announce@lists.debian.org list. This is a very low-traffic list (around a dozen messages a year), and only gives the most important announcements, such as the availability of a new stable release, the election of a new Project Leader, or the yearly Debian Conference.

➨ `https://lists.debian.org/debian-announce/`

More general (and regular) news about Debian are sent to the debian-news@lists.debian.org list. The traffic on this list is quite reasonable too (usually around a handful of messages a month), and it includes the semi-regular "Debian Project News", which is a compilation of various small bits of information about what happens in the project.

➨ https://lists.debian.org/debian-news/

COMMUNITY

The publicity team

Debian's official communication channels are managed by volunteers of the Debian publicity team. They are delegates of the Debian Project Leader and moderate news and announcements posted there. Many other volunteers contribute to the team, for example, by writing content for "Debian Project News", Debian's official blog (bits.debian.org[2]) or the microblogging service (micronews.debian.org[3]), which supplies social networking sites with microblogging content.

➨ https://wiki.debian.org/Teams/Publicity

For more information about the evolution of Debian and what is happening at some point in time in various teams, there is also the debian-devel-announce@lists.debian.org list. As its name implies, the announcements it carries will probably be more interesting to developers, but it also allows interested parties to keep an eye on what happens in more concrete terms than just when a stable version is released. While debian-announce@lists.debian.org gives news about the user-visible results, debian-devel-announce@lists.debian.org gives news about how these results are produced. As a side note, "d-d-a" (as it is sometimes referred to) is the only list that Debian developers must be subscribed to.

➨ https://lists.debian.org/debian-devel-announce/

Debian's official blog (bits.debian.org[4]) is also a good source of information. It conveys most of the interesting news that are published on the various mailing lists that we already covered and other important news contributed by community members. Since all Debian developers can contribute these news when they think they have something noteworthy to make public, Debian's blog gives a valuable insight while staying rather focused on the project as a whole.

A more informal source of information can also be found on Planet Debian, which aggregates articles posted by Debian contributors on their respective blogs. While the contents do not deal exclusively with Debian development, they provide a view into what is happening in the community and what its members are up to.

➨ https://planet.debian.org/

The project is also well represented on social networks. Debian only has an official presence on Identi.ca (microblogging platform, powered by *pump.io*), but there are some accounts retransmitting the RSS feed from https://micronews.debian.org/ and many Debian contributors who are posting on non-official accounts.

➨ https://identi.ca/debian

[2] https://bits.debian.org
[3] https://micronews.debian.org/
[4] https://bits.debian.org

- https://fosstodon.org/@debian
- https://twitter.com/debian
- https://www.facebook.com/debian
- https://www.flickr.com/groups/debian
- https://www.linkedin.com/company/debian

1.5. The Role of Distributions

A GNU/Linux distribution has two main objectives: install a free operating system on a computer (either with or without an existing system or systems), and provide a range of software covering all of the users' needs.

1.5.1. The Installer: `debian-installer`

The `debian-installer`, designed to be extremely modular in order to be as generic as possible, targets the first objective. It covers a broad range of installation situations and in general, greatly facilitates the creation of a derivative installer corresponding to a particular case.

This modularity, which also makes it very complex, may be daunting for the developers discovering this tool; but whether used in graphical or text mode, the user's experience is still similar. Great efforts have been made to reduce the number of questions asked at installation time, in particular thanks to the inclusion of automatic hardware detection software.

It is interesting to note that distributions derived from Debian differ greatly on this aspect, and provide a more limited installer (often confined to the i386 or amd64 architectures), but more user-friendly for the uninitiated. On the other hand, they usually refrain from straying too far from package contents in order to benefit as much as possible from the vast range of software offered without causing compatibility problems.

1.5.2. The Software Library

Quantitatively, Debian is undeniably the leader in this respect, with over 28,000 source packages. Qualitatively, Debian's policy and long testing period prior to releasing a new stable version justify its reputation for stability and consistency. As far as availability, everything is available on-line through many mirrors worldwide, with updates pushed out every six hours.

Many retailers sell DVD-ROMs on the Internet at a very low price (often at cost), the "images" for which are freely available for download. There is only one drawback: the low frequency of releases of new stable versions (their development sometimes takes more than two years), which delays the inclusion of new software.

Most new free software programs quickly find their way into the development version which allows them to be installed. If this requires too many updates due to their dependencies, the

program can also be recompiled for the stable version of Debian (see chapter 15, "Creating a Debian Package" page 448 for more information on this topic).

1.6. Lifecycle of a Release

The project will simultaneously have three to six different versions of each program, named *Experimental*, *Unstable*, *Testing*, *Stable*, *Oldstable*, and even *Oldoldstable*. Each one corresponds to a different phase in development. For a good understanding, let us take a look at a program's journey, from its initial packaging to inclusion in a stable version of Debian.

VOCABULARY
Release
The term "release", in the Debian project, indicates a particular version of a distribution (e.g., "unstable release" means "the unstable version"). It also indicates the public announcement of the launch of any new version (stable).

1.6.1. The *Experimental* Status

First let us take a look at the particular case of the *Experimental* distribution: this is a group of Debian packages corresponding to the software currently in development, and not necessarily completed, explaining its name. Not everything passes through this step; some developers add packages here in order to get feedback from more experienced (or braver) users.

Otherwise, this distribution frequently houses important modifications to base packages, whose integration into *Unstable* with serious bugs would have critical repercussions. It is, thus, a completely isolated distribution, its packages never migrate to another version (except by direct, express intervention of the maintainer or the ftpmasters). It is also not self-contained: only a subset of the existing packages are present in *Experimental*, and it generally does not include the base system. This distribution is therefore mostly useful in combination with another, self-contained, distribution such as *Unstable*.

1.6.2. The *Unstable* Status

Let us turn back to the case of a typical package. The maintainer creates an initial package, which they compile for the *Unstable* version and place on the ftp-master.debian.org server. This first event involves inspection and validation from the ftpmasters. The software is then available in the *Unstable* distribution, which is the "cutting edge" distribution chosen by users who are more concerned with having up-to-date packages than worried about serious bugs. They discover the program and then test it.

If they encounter bugs, they report them to the package's maintainer. The maintainer then regularly prepares corrected versions, which they upload to the server.

Every newly updated package is updated on all Debian mirrors around the world within six hours. The users then test the corrections and search for other problems resulting from the

modifications. Several updates may then occur rapidly. During these times, autobuilder robots come into action. Most frequently, the maintainer has only one traditional PC and has compiled their package on the amd64 (or i386) architecture (or they opted for a source-only upload, thus without any precompiled package); the autobuilders take over and automatically compile versions for all the other architectures. Some compilations may fail; the maintainer will then receive a bug report indicating the problem, which is then to be corrected in the next versions. When the bug is discovered by a specialist for the architecture in question, the bug report may come with a patch ready to use.

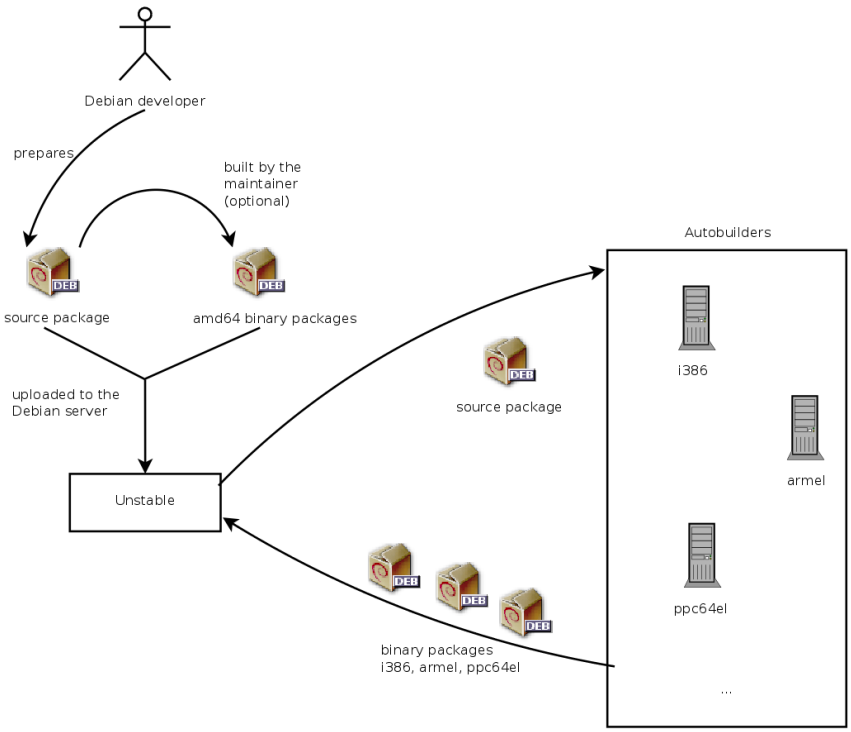


Figure 1.2 *Compilation of a package by the autobuilders*

QUICK LOOK

`buildd`, the Debian package recompiler

buildd is the abbreviation of "build daemon". This program automatically recompiles new versions of Debian packages on the architectures on which it is hosted (cross-compilation is avoided as much as possible).

Thus, to produce binaries for the `arm64` architecture, the project has `arm64` machines available. The *buildd* program runs on them continuously and creates binary packages for `arm64` from source packages sent by Debian developers.

This software is used on all the computers serving as autobuilders for Debian. By extension, the term *buildd* frequently is used to refer to these machines, which are generally reserved solely for this purpose.

1.6.3. Migration to *Testing*

A bit later, the package will have matured; compiled on all the architectures, it will not have undergone recent modifications. It is then a candidate for inclusion in the *Testing* distribution — a group of *Unstable* packages chosen according to some quantifiable criteria. Every day a program automatically selects the packages to include in *Testing*, according to elements guaranteeing a certain level of quality:

1. lack of critical bugs, or, at least fewer than the version currently included in *Testing*;
2. at least 5 days spent in *Unstable*, which is usually sufficient time to find and report any serious problems (successfully passing the package's own test suite, if it has one, reduces that time);
3. successful compilation on all officially supported architectures;
4. dependencies that can be satisfied in *Testing*, or that can at least be moved there together with the package in question;
5. automatic quality tests of the package (*autopkgtest*) — if defined — don't show any regression.

This system is clearly not infallible; critical bugs are regularly found in packages included in *Testing*. Still, it is generally effective, and *Testing* poses far fewer problems than *Unstable*, being for many, a good compromise between stability and novelty.

NOTE

Limitations of *Testing*

While very interesting in principle, *Testing* does have some practical problems: the tangle of cross-dependencies between packages is such that a package can rarely move there completely on its own. With packages all depending upon each other, it is sometimes necessary to migrate a large number of packages simultaneously, which is impossible when some are uploading updates regularly. On the other hand, the script identifying the families of related packages works hard to create them (this would be an NP-complete problem, for which, fortunately, we know some good heuristics). This is why we can manually interact with and guide this script by suggesting groups of packages, or imposing the inclusion of certain packages in a group, even if this temporarily breaks some dependencies. This functionality is accessible to the Release Managers and their assistants.

Recall that an NP-complete problem is of an exponential algorithmic complexity according to the size of the data, here being the length of the code (the number of figures) and the elements involved. The only way to resolve it is frequently to examine all possible configurations, which could require enormous means. A heuristic is an approximate, but satisfying, solution.

COMMUNITY

The Release Manager

Release Manager is an important title, associated with heavy responsibilities. The bearer of this title must, in effect, manage the release of a new, stable version of Debian, and define the process for development of *Testing* until it meets the quality criteria for *Stable*. They also define a tentative schedule (not always followed).

We also have Stable Release Managers, often abbreviated SRM, who manage and select updates for the current stable version of Debian. They systematically include security patches and examine all other proposals for inclusion, on a case by case basis, sent by Debian developers eager to update their package in the stable version.